

# A Framework for Area-efficient Multi-task BERT Execution on ReRAM-based Accelerators

Myeonggu Kang, Hyein Shin, Jaekang Shin, and Lee-Sup Kim

School of Electrical Engineering, Korea Advanced Institute of Science and Technology, Daejeon, Korea

{mgkang95, hyeinshin, jkshin94, leesup}@kaist.ac.kr

**Abstract**—With the superior algorithmic performances, BERT has become the *de-facto* standard model for various NLP tasks. Accordingly, multiple BERT models have been adopted on a single system, which is also called multi-task BERT. Although the ReRAM-based accelerator shows the sufficient potential to execute a single BERT model by adopting in-memory computation, processing multi-task BERT on the ReRAM-based accelerator extremely increases the overall area due to multiple fine-tuned models. In this paper, we propose a framework for area-efficient multi-task BERT execution on the ReRAM-based accelerator. Firstly, we decompose the fine-tuned model of each task by utilizing the base-model. After that, we propose a two-stage weight compressor, which shrinks the decomposed models by analyzing the properties of the ReRAM-based accelerator. We also present a profiler to generate hyper-parameters for the proposed compressor. By sharing the base-model and compressing the decomposed models, the proposed framework successfully reduces the total area of the ReRAM-based accelerator without an additional training procedure. It achieves a  $0.26\times$  area than baseline while maintaining the algorithmic performances.

## I. INTRODUCTION

The transformer-based language models [4], [12], [19] have become the *de-facto* standard model for various natural language processing (NLP) applications [2], [10], [15], [21]. Also, BERT, which brings the NLP's ImageNet moment, is the basis of the transformer-based language model. The unique training procedure of BERT, which consists of pre-training and fine-tuning, helps BERT understand the natural language well and outperform the conventional algorithms (*e.g.*, Seq2Seq [23]) on various NLP tasks. Despite its remarkable algorithmic performances, the larger model size and the lower operational intensity (ops/weight byte) of BERT cause the memory wall problem [26], where data traffic becomes the dominant portion of energy consumption and latency [14], [17], [22].

The ReRAM-based accelerators [3], [20], [27] are proposed to solve the data communication issue by adopting in-memory computation. They utilize the intrinsic property of ReRAM, where cell current is proportional to the applied voltage and cell conductance. Whole weight parameters are stored across ReRAM arrays in advance for the energy-efficient computation, and the computation is performed by applying input data to corresponding ReRAM arrays. By exploiting in-memory computation, the ReRAM-based accelerator enables energy-efficient execution of a single BERT model [29].

Despite its potential to process the single BERT model, executing multi-task BERT on the ReRAM-based accelerator induces a severe issue that exceedingly increases the overall

area. Multi-task BERT is defined as that multiple BERT models for various NLP tasks (*e.g.*, similarity checker, review classification) are processed on a single system. The training procedure of each multi-task BERT model consists of pre-training and fine-tuning stages. The fine-tuning process adjusts the base-model, which is generated in the pre-training stage, and it produces a corresponding fine-tuned model for each NLP task. Accordingly, as the number of NLP tasks to be supported in a single system increases, the number of fine-tuned models also increases in multi-task BERT. However, since the whole weight parameters should be programmed on ReRAM arrays in advance, a larger number of fine-tuned models exceedingly increase the total area on the ReRAM-based accelerator, which also decreases the utilization of computing resources.

Several works [5], [7], [11], [22], [30] have been proposed in the algorithm venue to solve the model size issue of multi-task BERT. However, as they do not consider the properties of the ReRAM-based accelerator, the methodologies in the previous works have limitations on the ReRAM-based accelerator. Firstly, the methodology in [30] requires fine-grained controls so it cannot be supported by the ReRAM-based accelerator, where the bulky MAC operation is performed. Also, although other methodologies [7], [11], [22], [30] can be adopted on the ReRAM-based accelerator, [11], [30] do not reduce the total area, or [7], [22] even increase the computation latency. Furthermore, all methods require an additional training process, which could hurt other training-based BERT optimizations [6], [18], [28]. Therefore, the proper solution to resolve the model size issue is mandatory for the ReRAM-based accelerator.

In this work, we propose a framework for area-efficient multi-task BERT execution on the ReRAM-based accelerator. We aim to achieve high area efficiency on the ReRAM-based accelerator without an additional training process. We observe that the fine-tuned model of each task is highly correlated to the base-model. Based on the observation, we first decompose the fine-tuned model by utilizing the base-model. After that, we propose a two-stage model compressor, which minimizes the size of the decomposed model by considering the characteristics of the ReRAM-based accelerator. We also present a profiler to generate hyper-parameters for the two-stage compressor. In the inference phase, the decomposed model and the base-model are merged together on the ReRAM-based accelerator. By sharing the base-model across multiple tasks and reducing the size of the decomposed models, the area-efficient multi-task BERT execution is realized on the ReRAM-based accelerator.

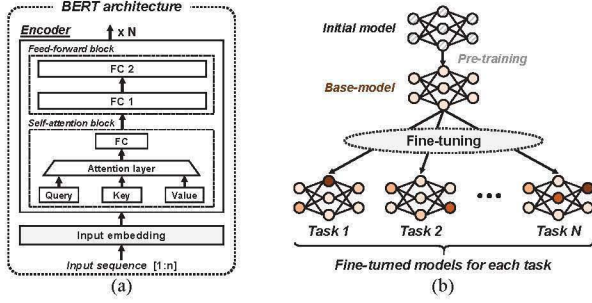


Fig. 1. (a) Model architecture, (b) training procedure of multi-task BERT.

The **major contributions** of this paper are as follows.

- We analyze the characteristics of multi-task BERT.
- We propose a framework to reduce the size of multi-task BERT by considering the properties of the ReRAM-based DNN accelerator. It also ensures algorithmic performances without the additional training process.
- We evaluate our framework on various base-models and datasets. Evaluation results show that the proposed work enables the area-efficient execution of multi-task BERT on the ReRAM-based accelerator.

## II. BACKGROUND AND MOTIVATION

### A. Multi-task BERT

With the superior algorithmic performances, the transformer-based language models (*e.g.*, BERT [4], RoBERTa [12], and DistilBERT [19]) have become the *de-facto* standard model for NLP applications. Among various transformer-based language models, BERT [4], which outperforms the human ability for the first time, is the basis of other models. Fig. 1 (a) describes the model architecture of BERT. Firstly, the input embedding layer converts input sequence words into corresponding word vectors, and they are passed to cascaded encoders. Each encoder includes self-attention block and feed-forward block. The self-attention block consists of fully-connected layers and attention computation, while the feed-forward block contains only fully-connected layers. The layer type forming each encoder is a fully-connected layer except the attention computation.

As BERT outperforms the conventional algorithms on various NLP tasks, multiple BERT models for various NLP tasks (*e.g.*, similarity checker, review classification) are adopted on a single system, which is known as multi-task BERT. The training of multi-task BERT consists of two stages: the pre-training and the fine-tuning for each task, which are shown in Fig. 1 (b). Firstly, the pre-training process trains the model to understand the natural language well. It generates the base-model by utilizing the large-scale unlabelled dataset. As the base-model comprehends general rules of the natural language, it can be widely utilized for various NLP tasks. The fine-tuning for each task is performed by utilizing the base-model. It tunes the base-model according to the task with a small-scale task-specific dataset. As a result, the fine-tuned model of each task is produced.

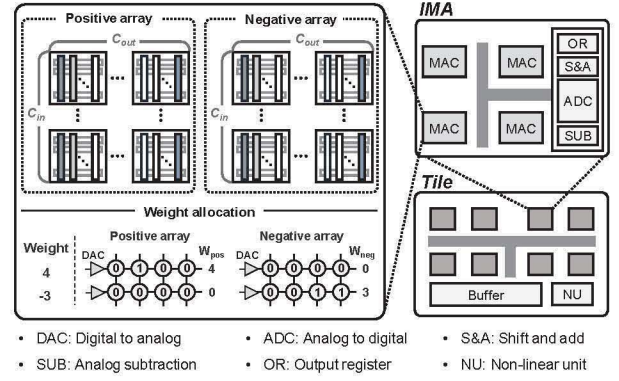


Fig. 2. An architecture of the ReRAM-based DNN accelerator.

### B. ReRAM-based DNN accelerator

The ReRAM-based DNN accelerators [3], [20], [27] have been proposed to execute DNN models efficiently by adopting in-memory computation. Fig. 2 shows the general architecture of the ReRAM-based accelerator. IMA is a unit processing element (PE), which contains ReRAM computing arrays. A weight value is represented by cells of positive and negative arrays ( $W_{pos}/W_{neg}$ ). According to the sign value and the cell resolution, the magnitude of weight value is stored in multiple cells of the corresponding array. Also, weight parameters of each output channel are allocated on the same bit-line. Whole weight parameters are programmed on ReRAM arrays in advance for the energy-efficient computation [3], [20], [27].

In the inference phase, the array-level MAC operation is executed by exploiting the intrinsic property of ReRAM, where the cell current is proportional to the applied voltage and the cell conductance. Firstly, input data are converted into analog voltage values via DAC and applied to the ReRAM array through word-lines. All cells in the array generate the cell current simultaneously, and each cell current represents a multiplication result of the input and weight data. The various cell current values are accumulated through a bit-line, and the analog subtraction between results of the positive and negative array is performed to generate partial sum results. The results are converted to a digital value via ADC. S&A block reconstructs a MAC operation output, and other non-linear functions are executed by a non-linear unit (NU) in each tile.

### C. Motivation

Although the ReRAM-based accelerator enables an energy-efficient execution of a single BERT model [29], processing multi-task BERT models on the ReRAM-based accelerator suffers from the severe issue that extremely increases the overall area. As described in Section II-A, the training process of multi-task BERT contains the fine-tuning process for each task, which produces the corresponding fine-tuned model. Thus, as a single system supports a larger number of NLP tasks, the system should hold a larger number of fine-tuned models [7], [11], [30]. A larger number of fine-tuned models exceedingly increase the total area on the ReRAM-based accelerator since whole BERT models should be programmed on ReRAM arrays in advance as described in Section II-B.

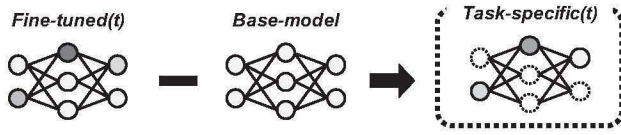


Fig. 3. Definition of task-specific(t).

To handle the model size issue, several works [5], [7], [11], [22], [30] have been presented for GPU. However, as previous works are based on GPU, the methodologies have limitations on the ReRAM-based accelerator. Firstly, [30] proposes the masking method, which requires fine-grained MAC operations. However, as the ReRAM-based accelerator performs the bulky MAC operation in the array-level, it cannot be supported by the ReRAM-based accelerator. Secondly, [5] generates irregular zero values in weight parameters, and [11] requires multiple BERT models. Thus, they do not reduce the total area on the ReRAM-based accelerator. The methods in [7], [22] even increase the computation latency on the ReRAM-based accelerator since they increase the number of fully-connected layers. Furthermore, all methodologies require an additional training process. However, the training procedure is expensive and it could hurt other training-based BERT optimizations (e.g., token-pruning [6], layer-dropping [18], early-exit [28]). Therefore, an appropriate solution to solve the model size issue should be proposed for the ReRAM-based accelerator without the additional training process.

### III. PROPOSED FRAMEWORK

We propose a framework that enables an area-efficient multi-task BERT execution on the ReRAM-based accelerator. The framework generates the compressed weight allocation of multi-task BERT by considering the characteristics of ReRAM-based accelerator. In the inference phase, each NLP task is performed with the compressed weight allocation. The observation and detailed descriptions of the framework are as follows.

#### A. Observation

The fine-tuning process of multi-task BERT is processed by utilizing the base-model and small-scale task-specific dataset. Accordingly, when fine-tuned(t) is defined as the fine-tuned model of task  $t$ , fine-tuned(t) is highly correlated to the base-model regardless of NLP tasks. We extract the correlation between the base-model and fine-tuned(t) by calculating cosine similarity, which is widely utilized to extract similarity between two vectors [1]. The cosine similarity is larger than 0.99 for all tasks as shown in Fig. 4 (a), and it indicates that the base-model and fine-tuned(t) are strongly correlated. When we define task-specific(t) by subtracting the base-model from fine-tuned(t) as shown in Fig. 3, task-specific(t) shows following aspects.

① Since fine-tuned(t) is highly correlated to the base-model, task-specific(t), which holds the difference between fine-tuned(t) and the base-model, has elements with small magnitude. Fig. 4 (b) shows non-zero elements ratio of task-specific(t). Task-specific(t) has 4%~40% of non-zero elements depending on the NLP task. Furthermore, the average magnitude of non-zero elements in task-specific(t) is only "1" regardless of

| Fine-tuned(t) | CoLA  | SST-2 | MRPC  | STS-B | QQP   | MNLI  | WNLI  | RTE   |
|---------------|-------|-------|-------|-------|-------|-------|-------|-------|
| Cosine sim.   | 0.999 | 0.997 | 0.999 | 0.999 | 0.992 | 0.991 | 0.999 | 0.999 |

◦ Cosine similarity becomes 1 when two vectors are identical (possible range: -1 ~ 1).

(a)

| Task-specific(t)   | CoLA | SST-2 | MRPC | STS-B | QQP  | MNLI | WNLI | RTE  |
|--------------------|------|-------|------|-------|------|------|------|------|
| Non-zero ratio (%) | 9.3  | 21.8  | 6.9  | 8.2   | 41.2 | 42.0 | 2.6  | 6.0  |
| Avg. magnitude     | 1.00 | 1.00  | 1.00 | 1.04  | 1.34 | 1.35 | 1.00 | 1.00 |

(b)

Fig. 4. (a) Cosine similarity between the base-model and fine-tuned(t) (feed-forward 1<sup>st</sup> of 5<sup>th</sup> encoder), (b) non-zero ratio and average magnitude of non-zero elements in task-specific(t) (base-model: BERT-base, model precision: 8-bit, dataset: GLUE, range of possible magnitude: 1~128).

| Task           | CoLA | SST-2 | MRPC | STS-B | QQP | MNLI | WNLI | RTE |
|----------------|------|-------|------|-------|-----|------|------|-----|
| Drop point (%) | 5    | >50   | 20   | 25    | 20  | 25   | >50  | 10  |

◦ We run 100 iterations with random binary mask, and the results are average of 100 iterations.

Fig. 5. Error resiliency of multi-task BERT (base-model: BERT-base, dataset: GLUE).

tasks (the range of possible magnitude: 1~128). Therefore, decomposing fine-tuned(t) as the combination of the base-model and task-specific(t) allows compressing task-specific(t) while maintaining the information of fine-tuned(t).

② Each NLP task (e.g., similarity checker, review classification) shows different task complexity. Accordingly, error resiliency is also different for each task. We test error resiliency of various NLP tasks by injecting errors to task-specific(t). The error model is set to binary mask with 0%~50% sparsity, and we mask the error model on task-specific(t). With defining a drop point as the minimum sparsity of the binary mask, which induces more than 1% accuracy drop from the baseline, Fig. 5 describes the drop point of various NLP tasks under the error resiliency test. Error-vulnerable tasks (CoLA, RTE) lose their algorithmic performances with small error injection, while error-resilient tasks (SST-2, WNLI) maintain the algorithmic performances despite huge error injection. The property gives an opportunity that each task can be compressed differently.

#### B. Two-stage model compressor

From the observations, we propose a two-stage model compressor, which generates compressed weight allocation (Fig. 7 (b)) of multi-task BERT without an additional training process. Fig. 6 describes an overall process of the two-stage compressor. We firstly decompose fine-tuned(t) as the combination of base-model and task-specific(t). After that, we reduce the bit-width and dimension of task-specific(t) by considering the properties of both ReRAM-based accelerator and multi-task BERT. When we compress task-specific(t), we utilize the cosine similarity as the major metric since the cosine similarity considers non-zero elements only, and it is independent of the vector length.

1) *Near-ternary quantization*: In the conventional ReRAM-based accelerator, a single weight value is divided into multiple cells according to the cell resolution (e.g., 1-/2- bit/cell), and the magnitude of weight value is stored in the positive or negative array depending on the sign value. Quantizing task-specific(t) into low-precision reduces the number of required cells, which results in high area-efficiency on the ReRAM-based accelerator. Also, task-specific(t) has elements with small magnitudes (average:  $\approx 1$ ) regardless of NLP tasks as described

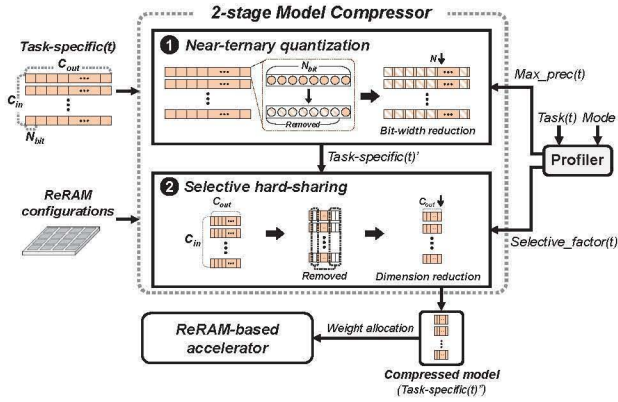


Fig. 6. Overview of two-stage model compressor.

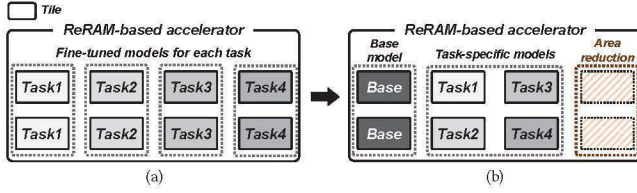


Fig. 7. (a) Initial weight allocation, (b) compressed weight allocation of multi-task BERT (Proposed work).

in Section III-A ①. The property gives an opportunity that task-specific( $t$ ) can be quantized into super low-precision without losing the information of task-specific( $t$ ). Therefore, we aim to reduce the bit-width of task-specific( $t$ ) to decrease the total number of required ReRAM cells in the near-ternary quantization process.

Fig. 8 describes an overall procedure of the near-ternary quantization process. It takes task-specific( $t$ ), ReRAM cell resolution and max\_prec( $t$ ), which is produced from the profiler, as inputs. ReRAM cell resolution becomes the unit quantization precision (e.g., 1/-2-bit) in the near ternary quantization since the number of ReRAM cells for weight parameters are determined by the cell resolution. Also, max\_prec( $t$ ) indicates an upper bound of quantization for each NLP task. The overall process is performed from the first layer to the last layer serially. Firstly, weight parameters of task-specific( $t$ ) are quantized to max\_prec( $t$ ) by the clipping method (quantization phase of iter 1 in Fig. 8 (b)), which makes the resolution of the quantized model the same as the original model. The elements having a larger magnitude than the maximum value of given precision are clipped to the maximum value. For example, the element "4" is clipped to "3" in the 2-bit precision, while other elements with "2" or "1" maintain the values as shown in Fig. 8 (a). Since the average magnitude of task-specific( $t$ ) is very tiny, the quantization by the clipping method can maintain the information of task-specific( $t$ ).

After the quantization phase, the assessment phase in Fig. 8 (a) is processed by utilizing the cosine similarity. If the cosine similarity between the quantized task-specific( $t$ ) and initial task-specific( $t$ ) is larger than 0.95, the assessment phase regards that the quantized model has the potential to be quantized to lower precision. Accordingly, the process performs the next

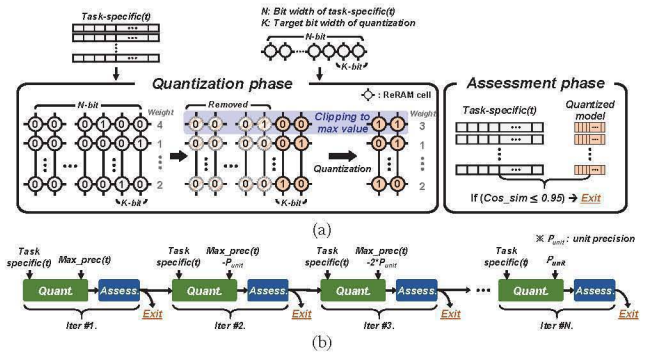


Fig. 8. (a) Quantization phase and assessment phase, (b) processing flow of near-ternary quantization.

iteration with decreasing quantization precision by the unit precision. The overall process is terminated when the cosine similarity is smaller than 0.95 as shown in Fig. 8 (b). As a result, task-specific( $t$ )' with smaller bit-width is generated.

2) **Selective hard-sharing**: The ReRAM-based accelerator performs the array-level computation, where all cells in an array produce cell currents simultaneously. Also, various current values in a bit-line are accumulated as described in Section II-B. Therefore, the value-level control is not feasible in the ReRAM-based accelerator, and eliminating the weight value without considering the structure of the ReRAM array does not enhance the area efficiency. Instead, removing output channels reduces the total area on the ReRAM-based accelerator since weight parameters of the same output channel are stored in the same bit-line as described in Section II-B. Therefore, we aim to eliminate output channels of task-specific( $t$ ) for further compression. Eliminating the output channel of task-specific( $t$ ) does not imply that fine-tuned( $t$ ) has zero elements for the removed output channels. Instead, fine-tuned( $t$ ) shares parameters of the base-model for the removed output channels (*hard-sharing*). Since fine-tuned( $t$ ) is highly correlated to the base-model as mentioned in Section III-A ①, hard-sharing the base-model for several output channels does not hurt the algorithmic performance.

Fig. 9 shows an overall procedure of the selective hard-sharing process. It takes task-specific( $t$ )' and selective\_factor( $t$ ) as inputs. Task-specific( $t$ )' is the output of near-ternary quantization, and selective\_factor( $t$ ) is the approximation bound of each task, which is generated from the profiler. Firstly, the overall procedure is performed layer-by-layer. For each layer of task-specific( $t$ )', we calculate the cost of each output channel by accumulating magnitudes of all elements in the output channel as Eq. (1):

$$Cost(k) = \sum_{i=1}^{N_e} (v_i^{pos}(k) + v_i^{neg}(k)) \quad (1)$$

where  $N_e$  is the number of elements in  $k^{th}$  output channel, and  $v_i^{pos}/v_i^{neg}$  is the value of each element stored in each positive/negative array. Since the cost is defined by accumulating the magnitude of elements in each output channel, the output channel with lower cost indicates that it does not include much

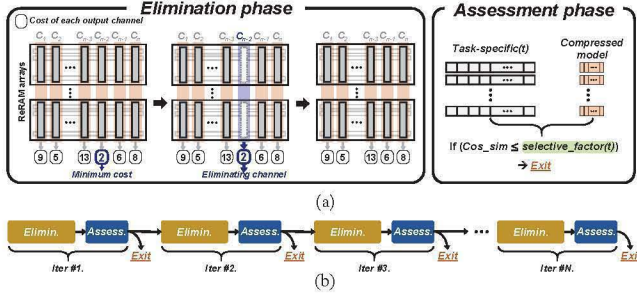


Fig. 9. (a) Elimination phase and assessment phase, (b) processing flow of selective hard-sharing.

information. Consequently, eliminating the output channel with a lower cost has less effect on the model than the output channel with a higher cost. Thus, we select the output channel with the minimum cost as Eq. (2):

$$(W_{k,cand.}^{pos}, W_{k,cand.}^{neg}) = \arg \min_k Cost(k) \quad (2)$$

where  $(W_{k,cand.}^{pos}, W_{k,cand.}^{neg})$  is a set of weight parameters in the output channel with the minimum cost. The elimination phase in Fig. 9 (a) removes  $(W_{k,cand.}^{pos}, W_{k,cand.}^{neg})$  from task-specific(t)' and produces task-specific(t)', which has fewer output channels. For example, the output channel  $C_{n-2}$  in Fig. 9 (a) has the minimum cost "2". Accordingly, the elimination phase removes  $C_{n-2}$  from task-specific(t)'. To evaluate the quality of task-specific(t)', the assessment phase is also performed by utilizing selective\_factor(t) and the cosine similarity. Suppose the cosine similarity between the initial task-specific(t) and task-specific(t)' is larger than selective\_factor(t). In that case, the assessment phase regards that more output channels can be removed from task-specific(t)'. Accordingly, the selective hard-sharing process continues eliminating the output channel. As described in Fig. 9 (b), the process is completed when the cosine similarity reaches selective\_factor(t). Finally, task-specific(t)', which has smaller bit-width and fewer output channels, is produced as the output of the two-stage model compressor.

### C. Profiler

The two-stage model compressor takes hyper-parameters ( $\max\_prec(t)$ ,  $\text{selective\_factor}(t)$ ) of each NLP task as inputs. In this section, we detail a profiler, which generates the hyper-parameters by considering the properties of NLP tasks. Algorithm 1 describes the overall process of the profiler.

$\max\_prec(t)$ , which is utilized as the global upper bound for all layers, is set to reduce the search space of the near-ternary quantization process. The profiler produces  $\max\_prec(t)$  for each NLP task, and Line 1-9 in Algorithm 1 describes an overall procedure to generate  $\max\_prec(t)$  of each task. Firstly, the quantization precision is initialized as the original precision. After that, all layers of task-specific(t) are quantized by adopting the clipping method. Suppose the cosine similarity between the quantized task-specific(t) and the initial task-specific(t) is larger than 0.95. In that case, the quantization process is performed again by decreasing the quantization precision by the unit precision. Finally, the profiler produces  $\max\_prec(t)$  of

### Algorithm 1: Profiler

**Input:** task-specific(t), ReRAM configuration, mode // indicates target task  
**Output:**  $\max\_prec(t)$ ,  $\text{selective\_factor}(t)$   
 // generate  $\max\_prec(t)$  for each task  
 1  $q\_prec, p\_unit = \text{original precision, ReRAM\_cell.resolution}$  // initialization  
 2 **while**  $q\_prec > 0$  **do**  
 3      $task\_specific(t) \leftarrow \text{Quant}(task\_specific(t), q\_prec, \text{clipping})$   
 4     **if**  $\text{Cosine\_sim}(task\_specific(t), task\_specific(t)) < 0.95$ :  
 5         **break**; // the process is finished by the assessment procedure  
 6     **end**  
 7      $q\_prec = q\_prec - p\_unit$  // decrease quantization precision by unit precision  
 8 **end**  
 9  $\max\_prec(t) = q\_prec$   
 // generate  $\text{selective\_factor}(t)$  for each task  
 10  $\text{selective\_list} = [1, 0.99, 0.95, 0.90, 0.85, 0.80, 0.75]$  // selective\_factor list for NLP tasks  
 11  $cl\_idx = \{\text{"CoLA, RTE"}: "0", \text{"MRPC, STS-B, QQP, MNLI"}: "1", \text{"SST-2, WNLI"}: "2"\}[t]$   
 12  $mode\_idx = \{\text{"acc"}: "0", \text{"opt"}: "2", \text{"agg"}: "4"\}[mode]$   
 13  $\text{selective\_factor}(t) = \text{selective\_list}[cl\_idx + mode\_idx]$   
 14 **Return**  $\max\_prec(t)$ ,  $\text{selective\_factor}(t)$

|                    | CoLA | SST-2 | MRPC | STS-B | QQP  | MNLI | WNLI | RTE  |
|--------------------|------|-------|------|-------|------|------|------|------|
| <b>Max_prec(t)</b> | 1    | 2     | 1    | 1     | 2    | 2    | 1    | 1    |
| (a)                |      |       |      |       |      |      |      |      |
|                    | CoLA | SST-2 | MRPC | STS-B | QQP  | MNLI | WNLI | RTE  |
| <b>Sel(t):Acc</b>  | 1    | 0.95  | 0.99 | 0.99  | 0.99 | 0.99 | 0.95 | 1    |
| <b>Sel(t):Opt</b>  | 0.95 | 0.85  | 0.90 | 0.90  | 0.90 | 0.90 | 0.85 | 0.95 |
| <b>Sel(t):Agg</b>  | 0.85 | 0.75  | 0.80 | 0.80  | 0.80 | 0.80 | 0.75 | 0.85 |
| (b)                |      |       |      |       |      |      |      |      |

Fig. 10. (a)  $\max\_prec(t)$ , (b)  $\text{selective\_factor}(t)$  of various NLP tasks (base-model: BERT-base, model precision: 8-bit, dataset: GLUE, ReRAM cell resolution: 1-bit/cell).

each NLP task for the near-ternary quantization process, and Fig. 10 (a) shows the  $\max\_prec(t)$  set of various tasks.

$\text{selective\_factor}(t)$  is the hyper-parameter for the assessment phase in the selective hard-sharing process. We consider both the error resiliency and user-defined mode: [acc, opt, agg] to generate  $\text{selective\_factor}(t)$ . The user-defined mode indicates that the user allows lower algorithmic performances for further area reduction. Line 10-13 in Algorithm 1 shows a procedure to produce  $\text{selective\_factor}(t)$ . Based on drop points in Fig. 5, the NLP tasks are clustered into 3 groups by K-means clustering [9]: C0-[CoLA, RTE], C1-[MRPC, STS-B, QQP, MNLI], and C2-[SST-2, WNLI]. Tasks in the larger cluster number are more error resilient than the smaller cluster number. Also, the aggressive mode (agg) allows the approximation by sacrificing algorithmic performances a bit. Therefore,  $\text{selective\_factor}(t)$  is assigned for each NLP task according to the cluster and the mode. For example, suppose that task and mode are set to "MRPC" and "opt", respectively. In that case,  $cl\_idx$  and  $mode\_idx$  become "1" and "2" (Line 11-12 in Algorithm 2), so we achieve  $\text{selective\_factor}(t)$  as "0.90", which is the element with the index "3" of the  $\text{selective\_list}$ . The  $\text{selective\_factor}(t)$  set of various tasks and modes is shown in Fig. 10 (b).

### D. Overall process

Algorithm 2 describes an overall procedure of the proposed framework, which generates the compressed weight allocation of multi-task BERT by utilizing the two-stage model compressor and the profiler. For each task in multi-task BERT, task-specific(t) is extracted by subtracting the base-model from fine-tuned(t) (Line 4 in Algorithm 2). The profiler produces the

### Algorithm 2: Overall process of the proposed framework

**Input:** Base-model, a set of fine-tuned models ( $t_1, t_2, \dots, t_n$ ),  
ReRAM configuration, mode

**Output:** Compressed weight allocation, a list of ( $\text{prec}(t)$ ,  $\text{removed}(t)$ )

```

1 compressed_list={}
2 meta_list={}
3 for each fine-tuned( $t_i$ ) do // fine-tuned( $t_i$ ) is the fine-tuned model of task i
4     task-specific( $t_i$ ) = fine-tuned( $t_i$ ) - base-model // decompose fine-tuned( $t_i$ )
5     // generate hyper-parameters of task i by utilizing the profiler
6     max_prec( $t_i$ ), selective_factor( $t_i$ ) = profiler(task-specific( $t_i$ ), ReRAM
7         configuration, mode)
8     // produce compressed task-specific( $t_i$ ) from the two-stage model compressor
9     task-specific( $t_i$ ), prec( $t_i$ ) = NT-Quant(task-specific( $t_i$ ), ReRAM
10         configuration, max_prec( $t_i$ ))
11     task-specific( $t_i$ ), removed( $t_i$ ) = Selective-HS(task-specific( $t_i$ ),
12         selective_factor( $t_i$ ))
13     compressed_list.append(task-specific( $t_i$ ))
14     meta_list.append((prec( $t_i$ ), removed( $t_i$ )))
15 end
16 Return compressed_list, meta_list

```

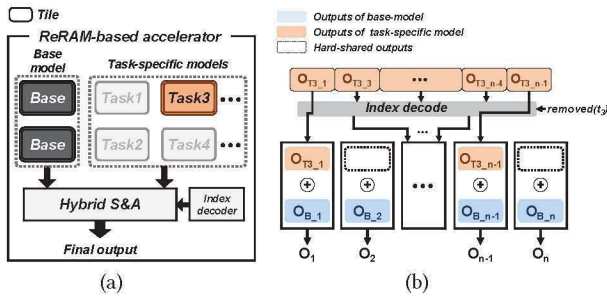


Fig. 11. (a) An example of executing NLP task on the proposed framework (target: Task 3), (b) summation process using hybrid S&A.

hyper-parameters of each task (Line 5 in Algorithm 2). After that, the two-stage model compressor generates a compressed weight allocation of each task-specific( $t$ ) by utilizing the hyper-parameters (Line 6-7 in Algorithm 2). Also, metadata about precision ( $\text{prec}(t)$ ) and index of removed output channels ( $\text{removed}(t)$ ) are obtained. Instead of programming multiple fine-tuned models for various NLP tasks (Fig. 7 (a)), the proposed framework stores one base-model and a set of compressed task-specific( $t$ ) on ReRAM arrays (Fig. 7 (b)). The base-model is shared across multiple NLP tasks to reduce the total area. The overall process of the proposed framework is performed once in the pre-processing phase, and it does not require the additional training procedure but utilizes fine-tuned models of multi-task BERT. In the inference phase, each NLP task is executed on the ReRAM-based accelerator by combining the base-model and compressed task-specific( $t$ ).

### E. Hardware support

As described in Section III-D, the base-model and the set of compressed task-specific( $t$ ) are programmed on ReRAM arrays. Since fine-tuned( $t$ ) is decomposed by the combination of base-model and task-specific( $t$ ), the ReRAM-based accelerator should add computation results of the base-model and task-specific( $t$ ) to reconstruct the final output. Simultaneously, the accelerator should consider the removed output channels of compressed task-specific( $t$ ). We modify the digital logic of the conventional ReRAM-based accelerator to implement the

TABLE I  
DESIGN CONFIGURATION TABLE

| Configuration | Description                                                               |
|---------------|---------------------------------------------------------------------------|
| Baseline      | Multi-task BERT execution without the proposed framework                  |
| $MT_{NQ}$     | Multi-task BERT execution with near-ternary quantization only             |
| $MT_{mode}$   | Multi-task BERT execution with the proposed framework (mode: acc,opt,agg) |

summation process as shown in Fig. 11. The summation process is performed in the global S&A stage, and it utilizes the obtained metadata for indexing logic. Also, the resolution of compressed task-specific( $t$ ) is identical to the base-model by adopting the clipping method. It enables the summation of each part without resolution matching. In conclusion, the proposed framework enables the area-efficient multi-task BERT execution on the ReRAM-based accelerator.

## IV. EVALUATION

### A. Evaluation methodology

1) *ReRAM-based accelerator*: We develop a cycle-level simulator of the ReRAM-based accelerator to evaluate our framework. The simulator is based on ISAAC [20] configuration with 32nm process node, and it utilizes positive and negative arrays to store weight parameters. The detailed configurations are as follows. The ReRAM array size is set to  $128 \times 128$ , and the cell resolution is 1-bit/cell. ADC/DAC resolutions are set to 8-/1-bit, respectively. The precision of both input and weight is 8-bit for the evaluation. We synthesize additional digital logic for the proposed framework using Synopsys Design Compiler with CMOS 65nm technology. We estimate the area and energy overhead of the additional hardware, and the logic is scaled to 32nm by following [8].

2) *Workloads*: We evaluate the proposed framework using BERT-base, which is provided by [25]. We utilize the GLUE dataset [24] (CoLA, SST-2, MRPC, STS-B, QQP, MNLI, WNLI, RTE). Since the proposed framework does not require an additional training procedure, the fine-tuned model of each task is used without any modification. The algorithmic performances of NLP tasks are obtained by Pytorch framework [16].

3) *Comparison baseline*: Table I describes design configurations. Baseline denotes executing multi-task BERT without the proposed framework.  $MT_{NQ}$  indicates processing multi-task BERT with the near-ternary quantization process.  $MT_{mode}$  is the proposed work that utilizes the near-ternary quantization and the selective hard-sharing together.  $MT_{mode}$  takes user-defined mode (acc, opt, agg), which allows lower algorithmic performances to further reduce the total area. We compare our work with [5], [7], [11], [22], [30] to validate the effectiveness of our methodology on the ReRAM-based accelerator. For further analysis, evaluation results on various cell resolutions and base-models are included in Section IV-E.

### B. Task-specific( $t$ ) on the ReRAM-based accelerator

Fig. 12 describes the average number of cells to express an element in task-specific( $t$ ) (1 indicates one cell in each

|            | CoLA                     | SST-2                    | MRPC                     | STS-B                    | QQP                      | MNLI                     | WNLI                     | RTE                      | Avg.                            |
|------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|--------------------------|---------------------------------|
| Baseline   | 8                        | 8                        | 8                        | 8                        | 8                        | 8                        | 8                        | 8                        | 8                               |
| $MT_{NQ}$  | 1<br>(0.13 $\times$ )    | 1.08<br>(0.14 $\times$ ) | 1<br>(0.13 $\times$ )    | 1<br>(0.13 $\times$ )    | 1.42<br>(0.18 $\times$ ) | 1.42<br>(0.18 $\times$ ) | 1<br>(0.13 $\times$ )    | 1<br>(0.13 $\times$ )    | <b>1.12</b><br>(0.14 $\times$ ) |
| $MT_{acc}$ | 1<br>(0.13 $\times$ )    | 0.95<br>(0.12 $\times$ ) | 0.97<br>(0.12 $\times$ ) | 0.97<br>(0.12 $\times$ ) | 1.40<br>(0.18 $\times$ ) | 1.40<br>(0.18 $\times$ ) | 0.85<br>(0.11 $\times$ ) | 1<br>(0.13 $\times$ )    | <b>1.07</b><br>(0.13 $\times$ ) |
| $MT_{opt}$ | 0.87<br>(0.11 $\times$ ) | 0.74<br>(0.09 $\times$ ) | 0.75<br>(0.09 $\times$ ) | 0.75<br>(0.09 $\times$ ) | 1.12<br>(0.14 $\times$ ) | 1.13<br>(0.14 $\times$ ) | 0.63<br>(0.08 $\times$ ) | 0.87<br>(0.11 $\times$ ) | <b>0.86</b><br>(0.11 $\times$ ) |
| $MT_{agg}$ | 0.67<br>(0.08 $\times$ ) | 0.56<br>(0.07 $\times$ ) | 0.56<br>(0.07 $\times$ ) | 0.56<br>(0.07 $\times$ ) | 0.86<br>(0.11 $\times$ ) | 0.87<br>(0.11 $\times$ ) | 0.46<br>(0.06 $\times$ ) | 0.66<br>(0.08 $\times$ ) | <b>0.65</b><br>(0.08 $\times$ ) |

Fig. 12. Average number of ReRAM cells for an element in task-specific(t).

|            | CoLA           | SST-2          | MRPC           | STS-B          | QQP            | MNLI           | WNLI        | RTE            | Score                 |
|------------|----------------|----------------|----------------|----------------|----------------|----------------|-------------|----------------|-----------------------|
| Baseline   | <b>58.9</b>    | <b>92.1</b>    | <b>86.8</b>    | <b>89.8</b>    | <b>90.8</b>    | <b>84.4</b>    | <b>56.3</b> | <b>70.4</b>    | <b>78.7</b>           |
| $MT_{NQ}$  | 58.9<br>(-)    | 92.1<br>(-)    | 86.8<br>(-)    | 89.8<br>(-)    | 90.8<br>(-)    | 84.4<br>(-)    | 56.3<br>(-) | 70.4<br>(-)    | 78.7<br>(-)           |
| $MT_{acc}$ | 58.9<br>(-)    | 92.4<br>(+0.3) | 87.0<br>(+0.2) | 89.8<br>(-)    | 90.7<br>(-0.1) | 84.5<br>(+0.1) | 56.3<br>(-) | 70.4<br>(-)    | <b>78.8</b><br>(+0.1) |
| $MT_{opt}$ | 57.6<br>(-1.3) | 92.3<br>(+0.2) | 87.3<br>(+0.5) | 89.7<br>(-0.1) | 88.4<br>(-2.4) | 82.5<br>(-1.9) | 56.3<br>(-) | 69.7<br>(-0.7) | <b>78.0</b><br>(-0.7) |
| $MT_{agg}$ | 56.3<br>(-2.6) | 92.3<br>(+0.2) | 86.5<br>(-0.3) | 88.7<br>(-1.1) | 81.8<br>(-9.0) | 78.4<br>(-6.0) | 56.3<br>(-) | 67.9<br>(-2.5) | <b>76.0</b><br>(-2.7) |

Fig. 13. Algorithmic performances of various design configurations.

positive and negative array). The baseline is the initial task-specific(t), which is decomposed from fine-tuned(t). Firstly, the near-ternary quantization process reduces the bit-width of task-specific(t), and most tasks are quantized to ternary precision. It directly reduces the number of required cells for task-specific(t).  $MT_{NQ}$  requires only 1.12 cells for task-specific(t) on average, and it is 0.14 $\times$  of baseline. Secondly, as the selective hard-sharing process eliminates output channels of task-specific(t),  $MT_{acc}$  achieves 0.95 $\times$  number of cells than  $MT_{NQ}$ . It requires only 1.07 cells on average. Furthermore, as the user-defined modes allow lower algorithmic performances,  $MT_{opt}$  and  $MT_{agg}$  achieve the smaller size of task-specific(t).  $MT_{opt}$  needs 0.86 cell for task-specific(t) on average, which is 0.77 $\times$  and 0.11 $\times$  of  $MT_{NQ}$  and baseline, respectively. Also,  $MT_{agg}$  requires only 0.65 cell for task-specific(t) on average, and it is 0.58 $\times$  and 0.08 $\times$  of  $MT_{NQ}$  and baseline, respectively. To conclude, the proposed framework successfully reduces the number of ReRAM cells for task-specific(t).

### C. Algorithmic performance

Fig. 13 shows algorithmic performances of various design configurations. We utilize BERT-base as a baseline. Firstly, although the near-ternary quantization reduces the bit-width of task-specific(t) into 1-/2-bit, the accuracy degradation does not occur in  $MT_{NQ}$ . In other words, the combination of high-precision base-model and low-precision task-specific(t) can provide identical algorithmic performances as the baseline. Also, the selective hard-sharing successfully reduces the dimension of task-specific(t) by considering the error resiliency of each NLP task. Both error-vulnerable tasks (CoLA, RTE) and error-resilient tasks (SST-2, WNLI) maintain the algorithmic performances in  $MT_{acc}$  with different selective\_factor(t). As the user-defined mode allows accuracy degradation to further reduce the total area,  $MT_{opt}$  and  $MT_{agg}$ , which produce more compressed task-specific(t) than  $MT_{acc}$ , provide 0.7 and 2.7 average score degradation from baseline, respectively. In conclusion, the proposed framework, which highly compresses task-specific(t), successfully maintains the algorithmic performance of each NLP task without the additional training procedure.

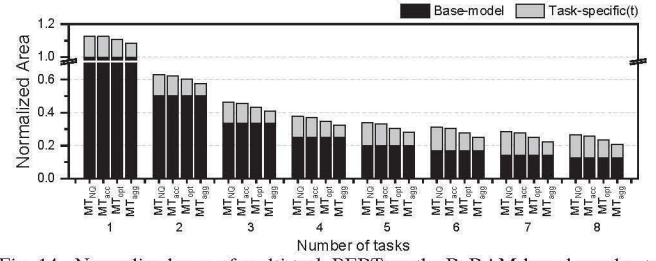


Fig. 14. Normalized area of multi-task BERT on the ReRAM-based accelerator.

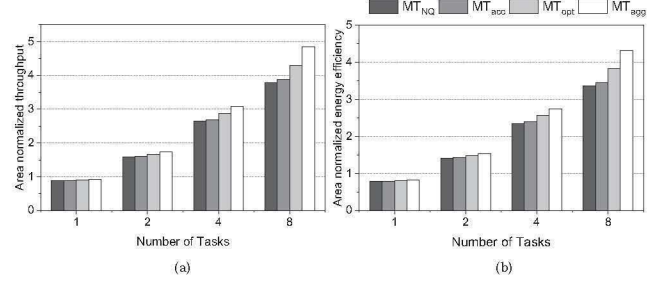


Fig. 15. Area normalized (a) throughput, (b) energy efficiency.

### D. Area

1) *Area*: Fig. 14 describes area reduction results on the ReRAM-based accelerator. Baseline utilizes multiple fine-tuned models, while the proposed framework uses one base-model and the set of compressed task-specific(t). We evaluate our framework on various number of tasks and NLP tasks are stacked from CoLA to RTE (e.g., 1: [CoLA], 3: [CoLA, SST-2, MRPC]). Evaluation results are normalized to baseline with each number of tasks. Firstly, since the proposed framework stores the base-model and task-specific(t) separately, it requires a larger area than baseline when the number of tasks is 1. Nevertheless, as the number of tasks increases, the proposed framework effectively reduces the total area from baseline by sharing the base-model across multiple tasks and compressing the set of task-specific(t). In detail, when the task number is 2, 4, and 8,  $MT_{acc}$  achieves 0.62 $\times$ , 0.37 $\times$ , and 0.26 $\times$  area than baseline, respectively.  $MT_{opt}$  and  $MT_{agg}$  reduce maximally 77% and 79% of total area. Furthermore, as the number of tasks increases, the portion of base-model in the proposed framework decreases. The base-model in  $MT_{acc}$  occupies 80% of the total area in the two tasks system while it accounts for about 50% in the eight tasks system. To conclude, the proposed work successfully reduces the total area of multi-task BERT on the ReRAM-based accelerator. Also, the area reductions increase in the system with a larger number of tasks.

2) *Throughput & energy efficiency*: Fig. 15 (a) shows area normalized throughput of the proposed work on the ReRAM-based accelerator. Firstly, the proposed work does not increase the latency since each computation of the base-model and task-specific(t) is performed in parallel. Therefore, the area reduction directly enhances the area normalized throughput.  $MT_{NQ}$  and  $MT_{acc}$  maximally achieve 3.8 $\times$  and 3.9 $\times$  higher area normalized throughput than baseline, respectively. Also, performance gains on  $MT_{agg}$  is up to 4.9 $\times$ .

Fig. 15 (b) describes the area normalized energy efficiency of the proposed framework. Since fine-tuned(t) is expressed

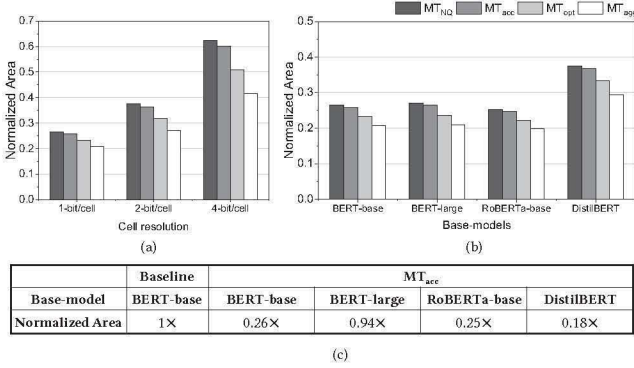


Fig. 16. Evaluation results on various (a) cell resolutions, (b) base-models (number of tasks: 8). (c) Area of the proposed work with various base-models (all results are normalized to baseline with BERT-base).

by the combination of the base-model and task-specific(t), the summation process, which merges the results of each part, is mandatory as described in Section III-E. Nevertheless, the computation amount of task-specific(t) is minimized from the compressed task-specific(t). As a result,  $MT_{NQ}$  and  $MT_{acc}$  achieve up to  $3.4\times$  and  $3.4\times$  higher area normalized energy efficiency over baseline, respectively.  $MT_{agg}$  shows maximally  $4.5\times$  higher gain over baseline.

### E. Further analysis

1) *Cell resolution*: Fig. 16 (a) describes evaluation results of the proposed framework on various cell resolutions. The near-ternary quantization process utilizes the cell resolution as the unit precision, and the average magnitude of task-specific(t) is about one for all NLP tasks. Therefore, a single cell is sufficient to express an element of task-specific(t) regardless of cell resolutions. Therefore, the area reduction of the proposed work is the largest in the low cell resolution, which is regarded as the practical ReRAM cell resolution [13].  $MT_{NQ}$  on 1-bit, 2-bit, and 4-bit resolutions achieve  $0.26\times$ ,  $0.38\times$ , and  $0.63\times$  area than baseline, respectively. Accordingly, the area reduction of 1-bit  $MT_{acc}$  is  $1.4\times$  and  $2.3\times$  larger than 2-bit and 4-bit resolutions, respectively.

2) *Base-model*: Fig. 16 (b) shows evaluation results of the proposed work on various base-models. We utilize BERT-large [4], RoBERTa-base [12], and DistilBERT [19] for the evaluation, and the results are normalized to baseline with each base-model. Since all base-models adopt the two-stage training procedure, the proposed framework successfully reduces the total area on the various base-models.  $MT_{acc}$  on BERT-large, RoBERTa-base, and DistilBERT reduce 74%, 76%, and 63% of total area from baseline, respectively. In conclusion, the proposed work can be generally adopted to reduce the total area for the various base-models.

Additionally, as each base-model shows different model configurations (e.g., layer number, hidden size), the multi-task system with each base-model shows a different total area. In detail,  $MT_{acc}$  on DistilBERT, which has the smallest base-model size among various models, requires the smallest area on the ReRAM-based accelerator as shown in Fig. 16 (c).

TABLE II  
COMPARISON WITH OTHER ALGORITHMS

|                            | [30] | [5] | [11] | [7] | [22] | Our work |
|----------------------------|------|-----|------|-----|------|----------|
| Without training?          | ✗    | ✗   | ✗    | ✗   | ✗    | ✓        |
| ReRAM structure-aware?     | ✗    | ✗   | ✗    | ✗   | ✗    | ✓        |
| Without fine-grained ctrl? | ✗    | ✓   | ✓    | ✓   | ✓    | ✓        |
| Area reduction?            | N/A  | ✗   | ✗    | ✓   | ✓    | ✓        |
| Latency maintenance?       | N/A  | ✓   | ✓    | ✗   | ✗    | ✓        |

† N/A indicates that the results are not available due to ReRAM incompatibility.

### F. Comparison with other algorithms

Table II summarizes the comparison with other algorithms [5], [7], [11], [22], [30]. Firstly, the proposed framework does not require the additional training procedure, which can hurt the other training-based BERT optimizations [6], [18], [28]. However, all methodologies in [5], [7], [11], [22], [30] are based on the training process. Secondly, the proposed work can reduce the total area while maintaining the computation latency on the ReRAM-based accelerator. However, the other methodologies have limitations on the ReRAM-based accelerator since they do not consider the characteristics of the ReRAM-based accelerator. In detail, [5], [11], which produces irregular zero elements or generates multiple fine-tuned models, do not reduce the area. Also, since each fully-connected layer becomes a pipeline stage with the same computation latency, [7], [22], which increase the number of fully-connected layers, increase the computation latency on the ReRAM-based accelerator. In conclusion, the proposed methodology is the most suitable approach to handle the model size issue on the ReRAM-based accelerator.

### G. Overhead analysis

The overall process of the proposed framework is performed once in the pre-processing phase. Therefore, it does not hurt the inference performances, and the pre-processing time can be amortized in many inferences. For the summation process between the base-model and task-specific(t), we modify the digital S&A block of the ReRAM-based accelerator. The modified S&A block accounts for less than 0.1% in total energy consumption and area, and the MAC operation for task-specific(t) is reduced from compressed task-specific(t). Two aspects minimize the overhead of the summation process.

## V. CONCLUSION

In this paper, we proposed the framework for the area-efficient multi-task BERT execution on the ReRAM-based accelerator. We analyzed the characteristics of multi-task BERT, and we decomposed the fine-tuned model of each task by utilizing the base-model. After that, we proposed the framework to compress the decomposed models by considering the properties of the ReRAM-based accelerator. By sharing one base-model and compressing decomposed models, the proposed framework reduced maximally 74% of the total area while maintaining the algorithmic performances.

## ACKNOWLEDGMENT

This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. NRF-2020R1A2B5B02002690), and Future Non-volatile Memory Cluster Academia Collaboration Program funded by Samsung Electronics.

## REFERENCES

- [1] R. Banner, I. Hubara, E. Hoffer, and D. Soudry, "Scalable methods for 8-bit training of neural networks," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18. Red Hook, NY, USA: Curran Associates Inc., 2018, p. 5151–5159.
- [2] Y. Belinkov and J. R. Glass, "Analysis methods in neural language processing: A survey," *CoRR*, vol. abs/1812.08951, 2018. [Online]. Available: <http://arxiv.org/abs/1812.08951>
- [3] P. Chi *et al.*, "Prime: A novel processing-in-memory architecture for neural network computation in rram-based main memory," in *Proc. ISCA*, 2016, pp. 27–39.
- [4] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," *CoRR*, vol. abs/1810.04805, 2018. [Online]. Available: <http://arxiv.org/abs/1810.04805>
- [5] M. A. Gordon, K. Duh, and N. Andrews, "Compressing bert: Studying the effects of weight pruning on transfer learning," 2020.
- [6] S. Goyal, A. R. Choudhury, S. M. Raje, V. T. Chakaravarthy, Y. Sabharwal, and A. Verma, "Power-bert: Accelerating bert inference via progressive word-vector elimination," 2020.
- [7] N. Houlsby, A. Giurugi, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, "Parameter-efficient transfer learning for nlp," 2019.
- [8] W. Huang, K. Rajamani, M. R. Stan, and K. Skadron, "Scaling with design constraints: Predicting the future of big chips," *IEEE Micro*, vol. 31, no. 4, pp. 16–29, 2011.
- [9] T. Kanungo, D. Mount, N. Netanyahu, C. Piatko, R. Silverman, and A. Wu, "An efficient k-means clustering algorithm: analysis and implementation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 24, no. 7, pp. 881–892, 2002.
- [10] K. Lee, M. Chang, and K. Toutanova, "Latent retrieval for weakly supervised open domain question answering," *CoRR*, vol. abs/1906.00300, 2019. [Online]. Available: <http://arxiv.org/abs/1906.00300>
- [11] X. Liu, P. He, W. Chen, and J. Gao, "Multi-task deep neural networks for natural language understanding," 2019.
- [12] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "Roberta: A robustly optimized BERT pretraining approach," *CoRR*, vol. abs/1907.11692, 2019. [Online]. Available: <http://arxiv.org/abs/1907.11692>
- [13] Y. Long, X. She, and S. Mukhopadhyay, "Design of reliable dnn accelerator with un-reliable rram," in *2019 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2019, pp. 1769–1774.
- [14] W. Niu, Z. Kong, G. Yuan, W. Jiang, J. Guan, C. Ding, P. Zhao, S. Liu, B. Ren, and Y. Wang, "Achieving real-time execution of transformer-based large-scale models on mobile with compiler-aware neural architecture optimization," 2020.
- [15] D. W. Otter, J. R. Medina, and J. K. Kalita, "A survey of the usages of deep learning in natural language processing," 2018.
- [16] A. Paszke *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019, pp. 8024–8035.
- [17] Y. Ren, S. Yoo, and A. Hoisie, "Performance analysis of deep learning workloads on leading-edge systems," 2019.
- [18] H. Sajjad, F. Dalvi, N. Durrani, and P. Nakov, "On the effect of dropping layers of pre-trained transformer models," 2021.
- [19] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter," 2019.
- [20] A. Shafiee *et al.*, "Isaac: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars," in *Proc. ISCA*, 2016, pp. 14–26.
- [21] L. B. Soares, N. FitzGerald, J. Ling, and T. Kwiatkowski, "Matching the blanks: Distributional similarity for relation learning," *CoRR*, vol. abs/1906.03158, 2019. [Online]. Available: <http://arxiv.org/abs/1906.03158>
- [22] Z. Sun, H. Yu, X. Song, R. Liu, Y. Yang, and D. Zhou, "Mobilebert: a compact task-agnostic bert for resource-limited devices," 2020.
- [23] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," 2014.
- [24] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman, "GLUE: A multi-task benchmark and analysis platform for natural language understanding," in *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*. Brussels, Belgium: Association for Computational Linguistics, Nov. 2018, pp. 353–355. [Online]. Available: <https://www.aclweb.org/anthology/W18-5446>
- [25] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Huggingface's transformers: State-of-the-art natural language processing," 2019.
- [26] W. A. Wulf and S. A. McKee, "Hitting the memory wall: Implications of the obvious," *SIGARCH Comput. Archit. News*, vol. 23, no. 1, p. 20–24, Mar. 1995. [Online]. Available: <https://doi.org/10.1145/216585.216588>
- [27] L. Xia, B. Li, T. Tang, P. Gu, P.-Y. Chen, S. Yu, Y. Cao, Y. Wang, Y. Xie, and H. Yang, "Mnsim: Simulation platform for memristor-based neuromorphic computing system," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 37, no. 5, pp. 1009–1022, 2018.
- [28] J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin, "Deebert: Dynamic early exiting for accelerating bert inference," 2020.
- [29] X. Yang, B. Yan, H. Li, and Y. Chen, "Retransformer: Rram-based processing-in-memory architecture for transformer acceleration," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–9.
- [30] M. Zhao, T. Lin, F. Mi, M. Jaggi, and H. Schütze, "Masking as an efficient alternative to finetuning for pretrained language models," 2020.